

TIAGO SANTOS

Lisbon, Portugal | tjmsantos07@gmail.com | [LinkedIn](#) | [GitHub](#) | [Portfolio](#)

Education

Instituto Superior Técnico

B.Sc. in Computer Science and Engineering

Lisbon, Portugal

Sept. 2025 – Expected June 2028

- **Relevant Coursework:** Object-Oriented Programming, Computer Architecture, Operating Systems

Technical Skills

Languages: C, RISC-V Assembly, Lua, TypeScript, JavaScript, Python

Frameworks: React

Databases: MySQL

Tools & Platforms: Git, Linux, Make, Valgrind, Logisim

Experience

Yoda Things

Co-Founder & Software Developer

Lisbon, Portugal

2024 – Present

- Co-founded and commercialized multiplayer game-server systems using Lua and MySQL, growing to 30+ paying clients.
- Built full-stack features across React/TypeScript, Lua server logic, and MySQL persistence for a live server supporting 100+ concurrent players.
- Designed 5+ event-driven Lua subsystems for state replication, player inventory, and in-game economy with MySQL-backed persistent state.
- Maintained 99%+ server uptime by profiling Lua scripts and optimizing MySQL indexes and queries to reduce thread-blocking during peak load.

Selected Projects

MiniTransformerGPT – Transformer LM & Custom Processor | *RISC-V Assembly, Logisim*

[\[Code\]](#)

- Implemented a bare-metal Transformer inference engine from scratch in RISC-V Assembly, including self-attention, matrix/vector operations, and argmax token selection.
- Designed a custom 16-bit processor in Logisim with dedicated hardware vector units to accelerate matrix operations for the model.
- Wrote RISC-V Assembly regression tests to validate self-attention and matrix-multiplication outputs against expected results.

In-Memory Logistics & Billing Engine | *C, Make, Valgrind*

[\[Code\]](#)

- Built a command-line invoicing and inventory management engine in C for product records, transaction tracking, and automated EAN validation.
- Implemented custom memory-allocation routines and in-memory data structures supporting 10,000+ product records with low memory overhead.
- Used Valgrind and stress testing to identify and fix memory leaks, achieving 0 errors and 0 leaks across the test suite.

Additional Information

Languages: Portuguese (Native), English (Fluent)